

2025-11-03-R-Final-Project-V2

Ei Phyu Sin Win, Htet Khant Linn, Khant Razar Kyaw, Khine Nwe Lin

2025-12-03

Library Installation

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidyr)
library(stringr)
library(lubridate)
library(readr)
library(ggplot2)
library(dplyr)
library(scales)
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##   discard
##
## The following object is masked from 'package:readr':
##
##   col_factor
```

```
library(forcats)
```

Data Loading

Loading each monthly sales

```
jan_sales <- read_csv("data/raw/2019-sales-monthly/201901-sales.csv")
```

```
## Rows: 9723 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(jan_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>             <chr>      <chr>
## 1 141234     iPhone          1              700        01/22/19 21~
## 2 141235     Lightning Charging Ca~ 1             14.95      01/28/19 14~
## 3 141236     Wired Headphones    2             11.99      01/17/19 13~
## 4 141237     27in FHD Monitor    1            149.99     01/05/19 20~
## 5 141238     Wired Headphones    1             11.99      01/25/19 11~
## 6 141239     AAA Batteries (4-pack) 1             2.99       01/29/19 20~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
feb_sales <- read_csv("data/raw/2019-sales-monthly/201902-sales.csv")
```

```
## Rows: 12036 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(feb_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>             <chr>      <chr>
## 1 150502     iPhone          1              700        02/18/19 01~
## 2 150503     AA Batteries (4-pack) 1             3.84       02/13/19 07~
## 3 150504     27in 4K Gaming Monitor 1            389.99     02/18/19 09~
## 4 150505     Lightning Charging Ca~ 1             14.95      02/02/19 16~
## 5 150506     AA Batteries (4-pack) 2             3.84       02/28/19 20~
## 6 150507     Lightning Charging Ca~ 1             14.95      02/24/19 18~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
mar_sales <- read_csv("data/raw/2019-sales-monthly/201903-sales.csv")
```

```
## Rows: 15226 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(mar_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>          <chr>             <chr>        <chr>
## 1 162009     iPhone             1               700          03/28/19 20~
## 2 162009     Lightning Charging Ca~ 1               14.95        03/28/19 20~
## 3 162009     Wired Headphones    2               11.99        03/28/19 20~
## 4 162010     Bose SoundSport Headp~ 1               99.99        03/17/19 05~
## 5 162011     34in Ultrawide Monitor 1               379.99       03/10/19 00~
## 6 162012     AA Batteries (4-pack) 1                3.84        03/20/19 21~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
apr_sales <- read_csv("data/raw/2019-sales-monthly/201904-sales.csv")
```

```
## Rows: 18383 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(apr_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>          <chr>             <chr>        <chr>
## 1 176558     USB-C Charging Cable  2               11.95        04/19/19 08~
## 2 <NA>      <NA>          <NA>             <NA>        <NA>
## 3 176559     Bose SoundSport Headp~ 1               99.99        04/07/19 22~
## 4 176560     Google Phone         1               600          04/12/19 14~
## 5 176560     Wired Headphones     1               11.99        04/12/19 14~
## 6 176561     Wired Headphones     1               11.99        04/30/19 09~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
may_sales <- read_csv("data/raw/2019-sales-monthly/201905-sales.csv")
```

```
## Rows: 16635 Columns: 6
## -- Column specification -----
```

```
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(may_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>              <chr>        <chr>
## 1 194095    Wired Headphones    1              11.99        05/16/19 17~
## 2 194096    AA Batteries (4-pack) 1              3.84        05/19/19 14~
## 3 194097    27in FHD Monitor    1             149.99       05/24/19 11~
## 4 194098    Wired Headphones    1              11.99        05/02/19 20~
## 5 194099    AAA Batteries (4-pack) 2              2.99        05/11/19 22~
## 6 194100    iPhone              1             700.0        05/10/19 19~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
jun_sales <- read_csv("data/raw/2019-sales-monthly/201906-sales.csv")
```

```
## Rows: 13622 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(jun_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>              <chr>        <chr>
## 1 209921    USB-C Charging Cable  1             11.95        06/23/19 19~
## 2 209922    Macbook Pro Laptop    1            1700.0       06/30/19 10~
## 3 209923    ThinkPad Laptop       1            999.99       06/24/19 20~
## 4 209924    27in FHD Monitor      1            149.99       06/05/19 10~
## 5 209925    Bose SoundSport Headp~ 1             99.99       06/25/19 18~
## 6 209926    Apple AirPods Headpho~ 1             150.0        06/28/19 20~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
jul_sales <- read_csv("data/raw/2019-sales-monthly/201907-sales.csv")
```

```
## Rows: 14371 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(jul_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>             <chr>       <chr>
## 1 222910    Apple AirPods Headph~ 1             150         07/26/19 16~
## 2 222911    Flatscreen TV         1             300         07/05/19 08~
## 3 222912    AA Batteries (4-pack) 1             3.84        07/29/19 12~
## 4 222913    AA Batteries (4-pack) 1             3.84        07/28/19 10~
## 5 222914    AAA Batteries (4-pack) 5             2.99        07/31/19 02~
## 6 222915    Bose SoundSport Headp~ 1             99.99       07/03/19 18~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
aug_sales <- read_csv("data/raw/2019-sales-monthly/201908-sales.csv")
```

```
## Rows: 12011 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(aug_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>             <chr>       <chr>
## 1 236670    Wired Headphones      2             11.99       08/31/19 22~
## 2 236671    Bose SoundSport Headp~ 1             99.99       08/15/19 15~
## 3 236672    iPhone                1             700.0       08/06/19 14~
## 4 236673    AA Batteries (4-pack) 2             3.84       08/29/19 20~
## 5 236674    AA Batteries (4-pack) 2             3.84       08/15/19 19~
## 6 236675    Wired Headphones      1             11.99       08/02/19 23~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
sep_sales <- read_csv("data/raw/2019-sales-monthly/201909-sales.csv")
```

```
## Rows: 11686 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(sep_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
```

```
##      <chr>      <chr>      <chr>      <chr>      <chr>
## 1 248151      AA Batteries (4-pack) 4      3.84      09/17/19 14~
## 2 248152      USB-C Charging Cable 2      11.95     09/29/19 10~
## 3 248153      USB-C Charging Cable 1      11.95     09/16/19 17~
## 4 248154      27in FHD Monitor      1      149.99    09/27/19 07~
## 5 248155      USB-C Charging Cable 1      11.95     09/01/19 19~
## 6 248156      34in Ultrawide Monitor 1      379.99    09/13/19 14~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
oct_sales <- read_csv("data/raw/2019-sales-monthly/201910-sales.csv")
```

```
## Rows: 20379 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(oct_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>      <chr>      <chr>
## 1 259358      34in Ultrawide Monitor 1      379.99      10/28/19 10~
## 2 259359      27in 4K Gaming Monitor 1      389.99      10/28/19 17~
## 3 259360      AAA Batteries (4-pack) 2      2.99        10/24/19 17~
## 4 259361      27in FHD Monitor      1      149.99      10/14/19 22~
## 5 259362      Wired Headphones      1      11.99       10/07/19 16~
## 6 259363      AAA Batteries (4-pack) 1      2.99        10/01/19 18~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
nov_sales <- read_csv("data/raw/2019-sales-monthly/201911-sales.csv")
```

```
## Rows: 17661 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(nov_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product      'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>      <chr>      <chr>      <chr>
## 1 278797      Wired Headphones      1      11.99      11/21/19 09~
## 2 278798      USB-C Charging Cable 2      11.95      11/17/19 10~
## 3 278799      Apple AirPods Headpho~ 1      150.0      11/19/19 14~
## 4 278800      27in FHD Monitor      1      149.99     11/25/19 22~
```

```
## 5 278801      Bose SoundSport Headp~ 1          99.99      11/09/19 13~
## 6 278802      USB-C Charging Cable  1          11.95      11/14/19 20~
## # i 1 more variable: 'Purchase Address' <chr>
```

```
dec_sales <- read_csv("data/raw/2019-sales-monthly/201912-sales.csv")
```

```
## Rows: 25117 Columns: 6
## -- Column specification -----
## Delimiter: ","
## chr (6): Order ID, Product, Quantity Ordered, Price Each, Order Date, Purcha...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
head(dec_sales)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product          'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>          <chr>             <chr>      <chr>
## 1 295665    Macbook Pro Laptop    1             1700      12/30/19 00:~
## 2 295666    LG Washing Machine    1             600.0     12/29/19 07:~
## 3 295667    USB-C Charging Cable  1             11.95     12/12/19 18:~
## 4 295668    27in FHD Monitor      1             149.99    12/22/19 15:~
## 5 295669    USB-C Charging Cable  1             11.95     12/18/19 12:~
## 6 295670    AA Batteries (4-pack) 1             3.84      12/31/19 22:~
## # i 1 more variable: 'Purchase Address' <chr>
```

Combine the 12 months data using bind_rows

```
all_sales_2019_combine <- bind_rows(jan_sales, feb_sales, mar_sales,
                                     apr_sales, may_sales, jun_sales,
                                     jul_sales, aug_sales, sep_sales,
                                     oct_sales, nov_sales, dec_sales)
```

```
head(all_sales_2019_combine)
```

```
## # A tibble: 6 x 6
##   'Order ID' Product          'Quantity Ordered' 'Price Each' 'Order Date'
##   <chr>      <chr>          <chr>             <chr>      <chr>
## 1 141234    iPhone          1             700       01/22/19 21~
## 2 141235    Lightning Charging Ca~ 1             14.95     01/28/19 14~
## 3 141236    Wired Headphones    2             11.99     01/17/19 13~
## 4 141237    27in FHD Monitor    1             149.99    01/05/19 20~
## 5 141238    Wired Headphones    1             11.99     01/25/19 11~
## 6 141239    AAA Batteries (4-pack) 1             2.99      01/29/19 20~
## # i 1 more variable: 'Purchase Address' <chr>
```

EDA Process

Clean the Data

Check the Table Info

```
glimpse(all_sales_2019_combine)
```

```
## Rows: 186,850
## Columns: 6
## $ 'Order ID'      <chr> "141234", "141235", "141236", "141237", "141238", "~
## $ Product         <chr> "iPhone", "Lightning Charging Cable", "Wired Headph~
## $ 'Quantity Ordered' <chr> "1", "1", "2", "1", "1", "1", "1", "1", "1", "1", "~
## $ 'Price Each'     <chr> "700", "14.95", "11.99", "149.99", "11.99", "2.99", ~
## $ 'Order Date'     <chr> "01/22/19 21:25", "01/28/19 14:15", "01/17/19 13:33~
## $ 'Purchase Address' <chr> "944 Walnut St, Boston, MA 02215", "185 Maple St, P~
```

Counts the Number of Missing Cells (NA)

```
total_NA_cells <- sum(is.na(all_sales_2019_combine))
cat("Total no. of missing cells: ", total_NA_cells, "\n")
```

```
## Total no. of missing cells: 3270
```

Counts the Number of Rows with at Least One NA

```
total_NA_rows <- sum(!complete.cases(all_sales_2019_combine))
cat("Total No. of missing rows: ", total_NA_rows, "\n")
```

```
## Total No. of missing rows: 545
```

Counting Total No. of Rows Before Dropping

```
row_before <- nrow(all_sales_2019_combine)
cat("Number of rows before dropping NA: ", row_before, "\n")
```

```
## Number of rows before dropping NA: 186850
```

Dropping NA Rows

```
all_sales_2019 <- drop_na(all_sales_2019_combine)
# counting total no. of rows after dropping
row_after <- nrow(all_sales_2019)
cat("Number of rows after dropping NA: ", row_after, "\n")
```



```
## Number of rows after dropping NA: 186305
```

```
# if this match with the total no. of missing row, we can validate it
cat("No. of dropped rows: ", row_before - row_after)
```

```
## No. of dropped rows: 545
```

Removing Duplicated Header Rows

```
all_sales_2019 <- all_sales_2019 %>%
  # this remove the no. of repeated header rows
  filter(`Order ID` != "Order ID") %>%
  mutate(
    OrderID = `Order ID`,
    QuantityOrdered = as.integer(`Quantity Ordered`),
    PriceEach = as.numeric(`Price Each`),
    OrderDate = `Order Date`,
    PurchaseAddress = `Purchase Address`,
    ZIP = str_extract(PurchaseAddress, "\\d{5}$")
  ) %>%
  select(OrderID, Product, QuantityOrdered, PriceEach, OrderDate, PurchaseAddress, ZIP)

row_after_cleaning_repeated_header <- nrow(all_sales_2019)

cat("Total no. of repeated header rows that was dropped: ", row_after - row_after_cleaning_repeated_header)
```

```
## Total no. of repeated header rows that was dropped: 355
```

Check the Table Info & Exporting

```
glimpse(all_sales_2019)
```

```
## Rows: 185,950
## Columns: 7
## $ OrderID      <chr> "141234", "141235", "141236", "141237", "141238", "141~
## $ Product      <chr> "iPhone", "Lightning Charging Cable", "Wired Headphone~
## $ QuantityOrdered <int> 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, ~
## $ PriceEach    <dbl> 700.00, 14.95, 11.99, 149.99, 11.99, 2.99, 389.99, 11.~
## $ OrderDate    <chr> "01/22/19 21:25", "01/28/19 14:15", "01/17/19 13:33", ~
## $ PurchaseAddress <chr> "944 Walnut St, Boston, MA 02215", "185 Maple St, Port~
## $ ZIP          <chr> "02215", "97035", "94016", "90001", "73301", "94016", ~
```

```
write_csv(all_sales_2019, "data/processed/all_sales_2019.csv")
```

Data Manipulation

Separating PurchaseAddress into 3 Columns (Street, City and State_ZIP)

```
all_sales_2019Splitted_and_formatted <- separate(all_sales_2019, PurchaseAddress, into = c("Street", "City", "ZIP"))
```

Add TotalSales & Separate City and ZIP columns

```
all_sales_2019Splitted_and_formatted <- all_sales_2019Splitted_and_formatted %>%
  mutate(
    TotalSales = QuantityOrdered * PriceEach,
    City = str_c(City, " (", str_extract(State_ZIP, "[A-Z]{2}"), ")"),
    ZIP = str_extract(State_ZIP, "\\d{5}$")
  )
```

Adding OrderDateTime, Month, and Hour Columns

```
all_sales_2019Splitted_and_formatted <- all_sales_2019Splitted_and_formatted %>%
  mutate(
    OrderDateTime = mdy_hm(OrderDate),
    Month = month(OrderDateTime, label = TRUE),
    Hour = hour(OrderDateTime)
  )
```

Select and Check the Clean Data

```
all_sales_2019Splitted_and_formatted <- all_sales_2019Splitted_and_formatted %>%
  select(
    OrderID, Product, QuantityOrdered, PriceEach, TotalSales, Street, City, ZIP, Month, Hour, OrderDateTime
  )

glimpse(all_sales_2019Splitted_and_formatted)
```

```
## Rows: 185,950
## Columns: 11
## $ OrderID      <chr> "141234", "141235", "141236", "141237", "141238", "141~
## $ Product      <chr> "iPhone", "Lightning Charging Cable", "Wired Headphone~
## $ QuantityOrdered <int> 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, ~
## $ PriceEach    <dbl> 700.00, 14.95, 11.99, 149.99, 11.99, 2.99, 389.99, 11.~
## $ TotalSales   <dbl> 700.00, 14.95, 23.98, 149.99, 11.99, 2.99, 389.99, 11.~
## $ Street       <chr> "944 Walnut St", "185 Maple St", "538 Adams St", "738 ~
## $ City         <chr> "Boston (MA)", "Portland (OR)", "San Francisco (CA)", ~
## $ ZIP          <chr> "02215", "97035", "94016", "90001", "73301", "94016", ~
## $ Month        <ord> Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, Jan, ~
## $ Hour         <int> 21, 14, 13, 20, 11, 20, 12, 12, 10, 21, 11, 10, 18, 19~
## $ OrderDateTime <dtm> 2019-01-22 21:25:00, 2019-01-28 14:15:00, 2019-01-17 ~
```

Exporting Clean Data into CSV

```
# write_csv(all_sales_2019_splitting_and_formatted, "data/processed/all_sales_2019_v2.csv")
```

EDA Process

Importing Data

```
all_sales_2019 <- read_csv("data/processed/all_sales_2019_v2.csv")
```

```
## Rows: 185950 Columns: 11
## -- Column specification -----
## Delimiter: ","
## chr (5): Product, Street, City, ZIP, Month
## dbl (5): OrderID, QuantityOrdered, PriceEach, TotalSales, Hour
## dtm (1): OrderDateTime
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# Check data structure and Null Value.
head(all_sales_2019)
```

```
## # A tibble: 6 x 11
##   OrderID Product QuantityOrdered PriceEach TotalSales Street City ZIP Month
##   <dbl> <chr>          <dbl>      <dbl>      <dbl> <chr> <chr> <chr> <chr>
## 1  141234 iPhone              1      700        700  944 W~ Bost~ 02215 Jan
## 2  141235 Lightni~          1     15.0      15.0  185 M~ Port~ 97035 Jan
## 3  141236 Wired H~          2     12.0      24.0  538 A~ San ~ 94016 Jan
## 4  141237 27in FH~          1     15.0      15.0  738 1~ Los ~ 90001 Jan
## 5  141238 Wired H~          1     12.0      12.0  387 1~ Aust~ 73301 Jan
## 6  141239 AAA Bat~          1      2.99      2.99  775 W~ San ~ 94016 Jan
## # i 2 more variables: Hour <dbl>, OrderDateTime <dtm>
```

```
glimpse(all_sales_2019)
```

```
## Rows: 185,950
## Columns: 11
## $ OrderID      <dbl> 141234, 141235, 141236, 141237, 141238, 141239, 141240~
## $ Product      <chr> "iPhone", "Lightning Charging Cable", "Wired Headphone~
## $ QuantityOrdered <dbl> 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, ~
## $ PriceEach    <dbl> 700.00, 14.95, 11.99, 149.99, 11.99, 2.99, 389.99, 11.~
## $ TotalSales   <dbl> 700.00, 14.95, 23.98, 149.99, 11.99, 2.99, 389.99, 11.~
## $ Street       <chr> "944 Walnut St", "185 Maple St", "538 Adams St", "738 ~
## $ City         <chr> "Boston (MA)", "Portland (OR)", "San Francisco (CA)", ~
## $ ZIP          <chr> "02215", "97035", "94016", "90001", "73301", "94016", ~
## $ Month        <chr> "Jan", "Jan", "Jan", "Jan", "Jan", "Jan", "Jan", "Jan"~
## $ Hour         <dbl> 21, 14, 13, 20, 11, 20, 12, 12, 10, 21, 11, 10, 18, 19~
## $ OrderDateTime <dtm> 2019-01-22 21:25:00, 2019-01-28 14:15:00, 2019-01-17 ~
```

Calculating Summary Statistics

```
cat("=== General Summary of Data ===\n")
```

```
## === General Summary of Data ===
```

```
summary(all_sales_2019 %>% select(QuantityOrdered, PriceEach, TotalSales))
```

```
##  QuantityOrdered  PriceEach      TotalSales
##  Min.   :1.000    Min.      :  2.99    Min.      :  2.99
##  1st Qu.:1.000    1st Qu.: 11.95    1st Qu.: 11.95
##  Median :1.000    Median : 14.95    Median : 14.95
##  Mean   :1.124    Mean     :184.40    Mean     :185.49
##  3rd Qu.:1.000    3rd Qu.:150.00    3rd Qu.:150.00
##  Max.   :9.000    Max.     :1700.00   Max.     :3400.00
```

Checking Null Value

```
sum(is.na(all_sales_2019))
```

```
## [1] 0
```

Question 1: What was the best month for sales? How much was earned that month?

```
total_sales_each_month <- all_sales_2019 %>%
  group_by(Month) %>%
  summarize(TotalSales = sum(TotalSales)) %>%
  arrange(desc(TotalSales))

# write_csv(total_sales_each_month, "outputs/tables/total_sales_each_month.csv")

month_order = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")

# ordering month for visual
total_sales_each_month <- total_sales_each_month %>%
  mutate(Month = factor(Month, levels = month_order))

vis_1 <- ggplot(total_sales_each_month, aes(x = Month, y = TotalSales)) +
  geom_col() +
  labs(title = "Total Sales by Month",
       x = "Month",
       y = "Total Sales in USD ($)")

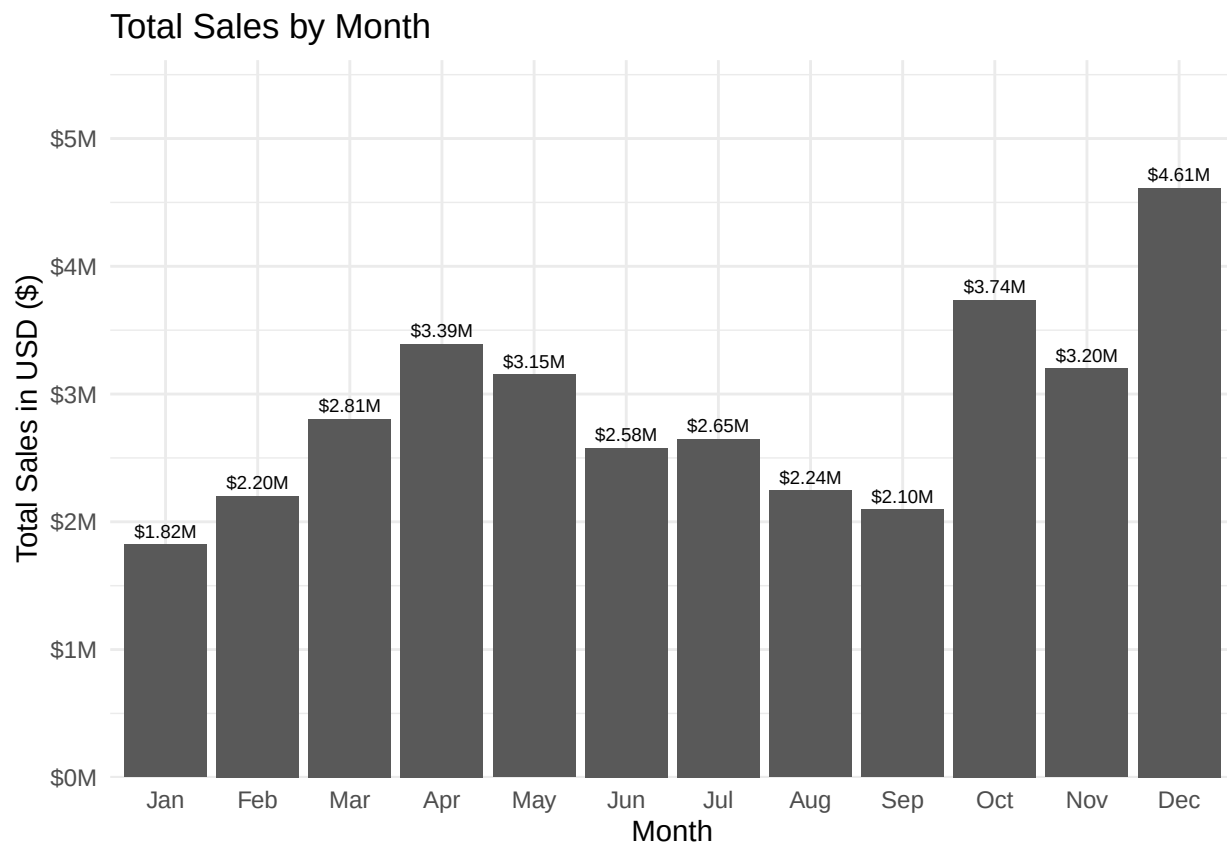
vis_1 <- vis_1 +
```

```

geom_text(aes(label = scales::dollar(TotalSales, scale = 1/1e6, suffix = "M")),
          vjust = -0.5,
          size = 2.5) +
scale_y_continuous(
  labels = scales::dollar_format(scale = 1/1e6, suffix = "M"),
  expand = expansion(add = c(0, 1e6))
) +
theme_minimal()

# ggsave("outputs/figures/total_sales_by_month.jpg", plot = vis_1)
print(vis_1)

```



```

best_month_for_sales <- total_sales_each_month %>%
  slice_max(TotalSales, n = 1)

cat("The best month for sales is", best_month_for_sales$Month,
    "with the total sales of",
    paste0("$", best_month_for_sales$TotalSales, "."),
    "\n")

```

```
## The best month for sales is 12 with the total sales of $4613443.34.
```

Question 2: What city has the highest sales?

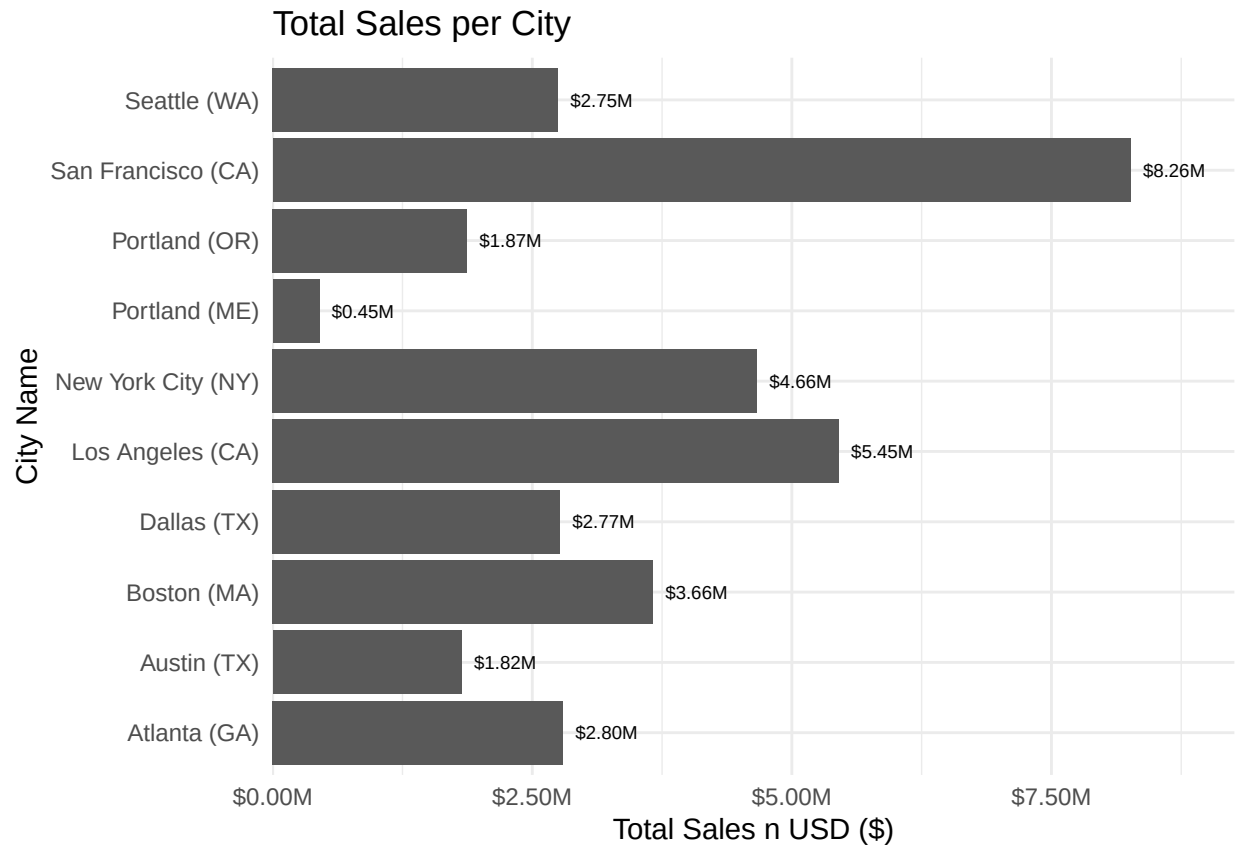
```
total_sales_each_city <- all_sales_2019 %>%
  group_by(City) %>%
  summarize(TotalSales = sum(TotalSales)) %>%
  arrange(desc(TotalSales))

# write_csv(total_sales_each_city, "outputs/tables/total_sales_each_city.csv")

vis_2 <- ggplot(total_sales_each_city, aes(TotalSales, City)) +
  geom_col()

vis_2 <- vis_2 +
  labs(
    title = "Total Sales per City",
    x = "Total Sales n USD ($)",
    y = "City Name"
  ) +
  geom_text(aes(label = scales::dollar(TotalSales, scale = 1/1e6, suffix = "M")),
            hjust = -0.2,
            size = 2.5) +
  scale_x_continuous(
    label = scales::dollar_format(scale = 1/1e6, suffix = "M"),
    expand = expansion(add = c(0, 1e6))
  ) +
  theme_minimal()

# ggsave("outputs/figures/total_sales_per_city.jpg", plot = vis_2)
print(vis_2)
```



```
city_with_highest_sales <- total_sales_each_city %>%
  slice_max(TotalSales, n = 1)

cat("The city with highest sales is", city_with_highest_sales$City,
    "with the total sales of",
    paste0("$", city_with_highest_sales$TotalSales, "."),
    "\n")
```

```
## The city with highest sales is San Francisco (CA) with the total sales of $8262203.91.
```

Question 3: What time should we display advertisements to maximize likelihood of customers buying products?

```
glimpse(all_sales_2019)
```

```
## Rows: 185,950
## Columns: 11
## $ OrderID      <dbl> 141234, 141235, 141236, 141237, 141238, 141239, 141240~
## $ Product      <chr> "iPhone", "Lightning Charging Cable", "Wired Headphone~
## $ QuantityOrdered <dbl> 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, ~
## $ PriceEach    <dbl> 700.00, 14.95, 11.99, 149.99, 11.99, 2.99, 389.99, 11.~
```

```
## $ TotalSales      <dbl> 700.00, 14.95, 23.98, 149.99, 11.99, 2.99, 389.99, 11.~
## $ Street          <chr> "944 Walnut St", "185 Maple St", "538 Adams St", "738 ~
## $ City            <chr> "Boston (MA)", "Portland (OR)", "San Francisco (CA)", ~
## $ ZIP             <chr> "02215", "97035", "94016", "90001", "73301", "94016", ~
## $ Month           <chr> "Jan", "Jan", "Jan", "Jan", "Jan", "Jan", "Jan", "Jan"~
## $ Hour            <dbl> 21, 14, 13, 20, 11, 20, 12, 12, 10, 21, 11, 10, 18, 19~
## $ OrderDateTime   <dtm> 2019-01-22 21:25:00, 2019-01-28 14:15:00, 2019-01-17 ~
```

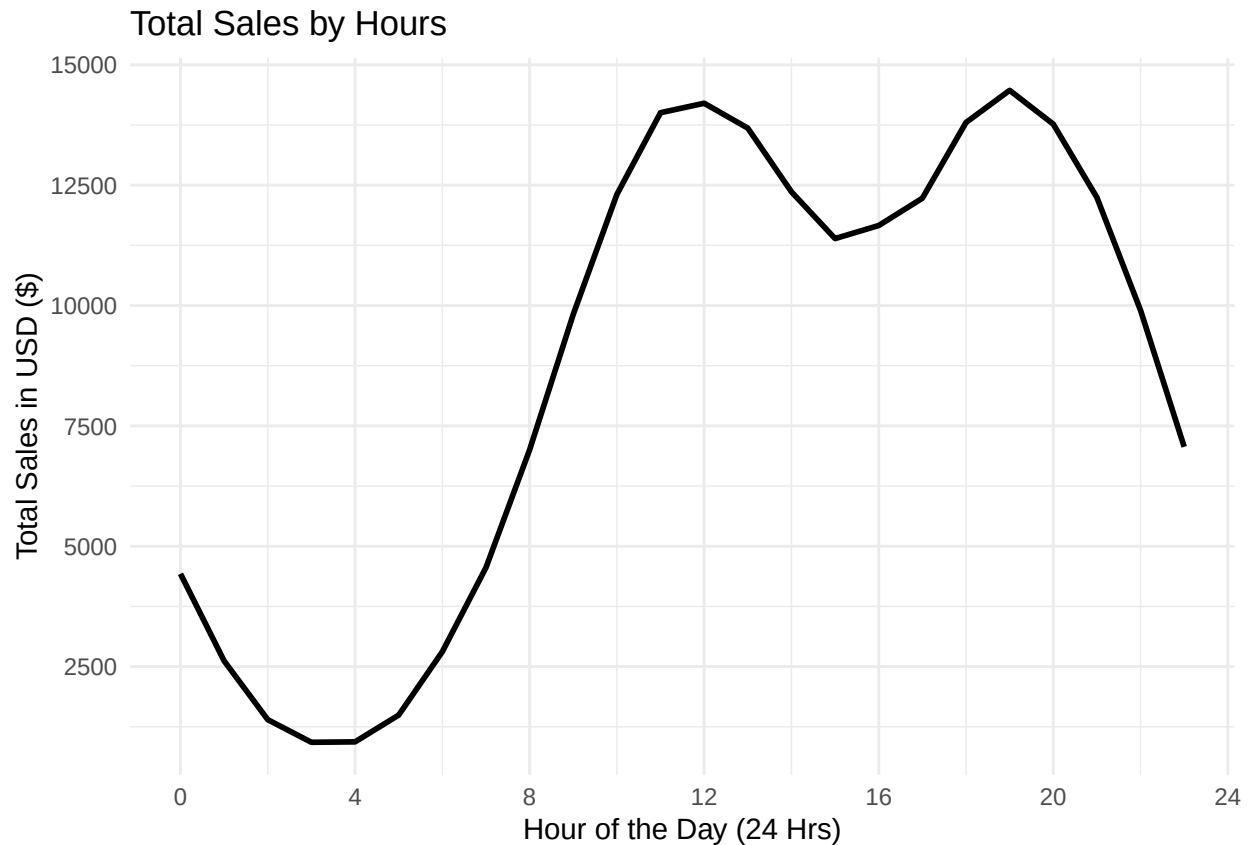
```
sales_by_hours <- all_sales_2019 %>%
  group_by(Hour) %>%
  summarize(TotalQuantitySales = sum(QuantityOrdered))

# write_csv(sales_by_hours, "outputs/tables/sales_by_hours.csv")

vis_3 <- ggplot(sales_by_hours, aes(x = Hour, y = TotalQuantitySales)) +
  geom_line(size = 1) +
  labs(
    title = "Total Sales by Hours",
    y = "Total Sales in USD ($)",
    x = "Hour of the Day (24 Hrs)"
  ) +
  scale_x_continuous(breaks = seq(0, 24, by = 4)) +
  scale_y_continuous(breaks = seq(0, 17500, by = 2500)) +
  theme_minimal()
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
# ggsave("outputs/figures/total_sales_by_hours.jpg", plot = vis_3)
print(vis_3)
```

Question 4: What products are most often sold together?

```
duplicated_OrderID <- all_sales_2019 %>%
  group_by(OrderID) %>%
  filter(n() > 1) %>%
  select(OrderID, Product) %>%
  ungroup()

products_sold_together <- duplicated_OrderID %>%
  group_by(OrderID) %>%
  # count(sort = TRUE) %>%
  summarize(ProductPairs = paste(sort(Product), collapse = ", ")) %>%
  ungroup()

products_sold_together_count <- products_sold_together %>%
  count(ProductPairs, sort = TRUE) %>%
  rename(TotalSales = n)

top_10_products_sold_together <- head(products_sold_together_count, n = 10)

print(top_10_products_sold_together)
```

```
## # A tibble: 10 x 2
```

##	ProductPairs	TotalSales
##	<chr>	<int>
## 1	iPhone, Lightning Charging Cable	891
## 2	Google Phone, USB-C Charging Cable	868
## 3	iPhone, Wired Headphones	374
## 4	USB-C Charging Cable, Vareebadd Phone	318
## 5	Google Phone, Wired Headphones	311
## 6	Apple AirPods Headphones, iPhone	299
## 7	Bose SoundSport Headphones, Google Phone	169
## 8	Vareebadd Phone, Wired Headphones	110
## 9	AA Batteries (4-pack), Lightning Charging Cable	103
## 10	Lightning Charging Cable, USB-C Charging Cable	96

```
# write_csv(products_sold_together_count, "outputs/tables/products_sold_together.csv")
```

Question 5: What product sold the most?

```
single_product_sold <- all_sales_2019 %>%
  group_by(Product) %>%
  count(wt = QuantityOrdered, sort = TRUE) %>%
  rename(TotalUnitSold = n)
```

```
most_single_product_sold <- single_product_sold %>%
  head(1)
```

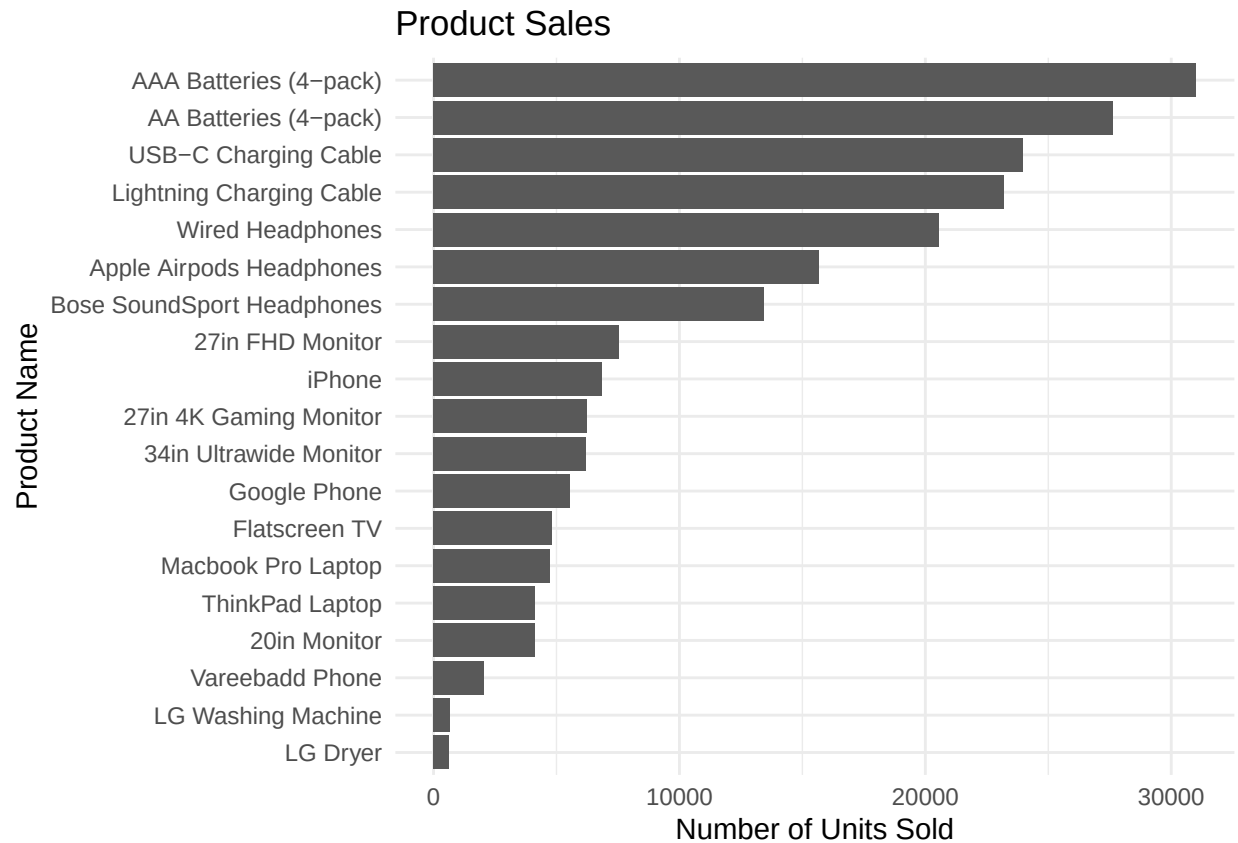
```
cat(paste0("The most sold product is ", most_single_product_sold$Product,
  " with the total sales of ", most_single_product_sold$TotalUnitSold,
  " items. \n"))
```

```
## The most sold product is AAA Batteries (4-pack) with the total sales of 31017 items.
```

```
# write_csv(single_product_sold, "outputs/tables/product_sales.csv")
```

```
vis_4 <- ggplot(single_product_sold, aes(TotalUnitSold, fct_reorder(Product, TotalUnitSold))) +
  geom_col() +
  labs(
    title = "Product Sales",
    x = "Number of Units Sold",
    y = "Product Name"
  ) +
  theme_minimal()
```

```
# ggsave("outputs/figures/product_sales.jpg", plot = vis_4)
print(vis_4)
```



Building Time series model

```
library(tidyverse)
library(lubridate)
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
```

```
library(tseries)
```

```
# all_sales_2019 <- read_csv("data/processed/all_sales_2019_v2.csv")
```

aggregate sales by day

```
daily_sales <- all_sales_2019 %>%
  mutate(Date = as.Date(OrderDateTime)) %>% # Date format
  group_by(Date) %>%
  summarize(TotalSales = sum(TotalSales)) %>%
  arrange(Date) # data is ordered chronologically
```

create a Time Series Object

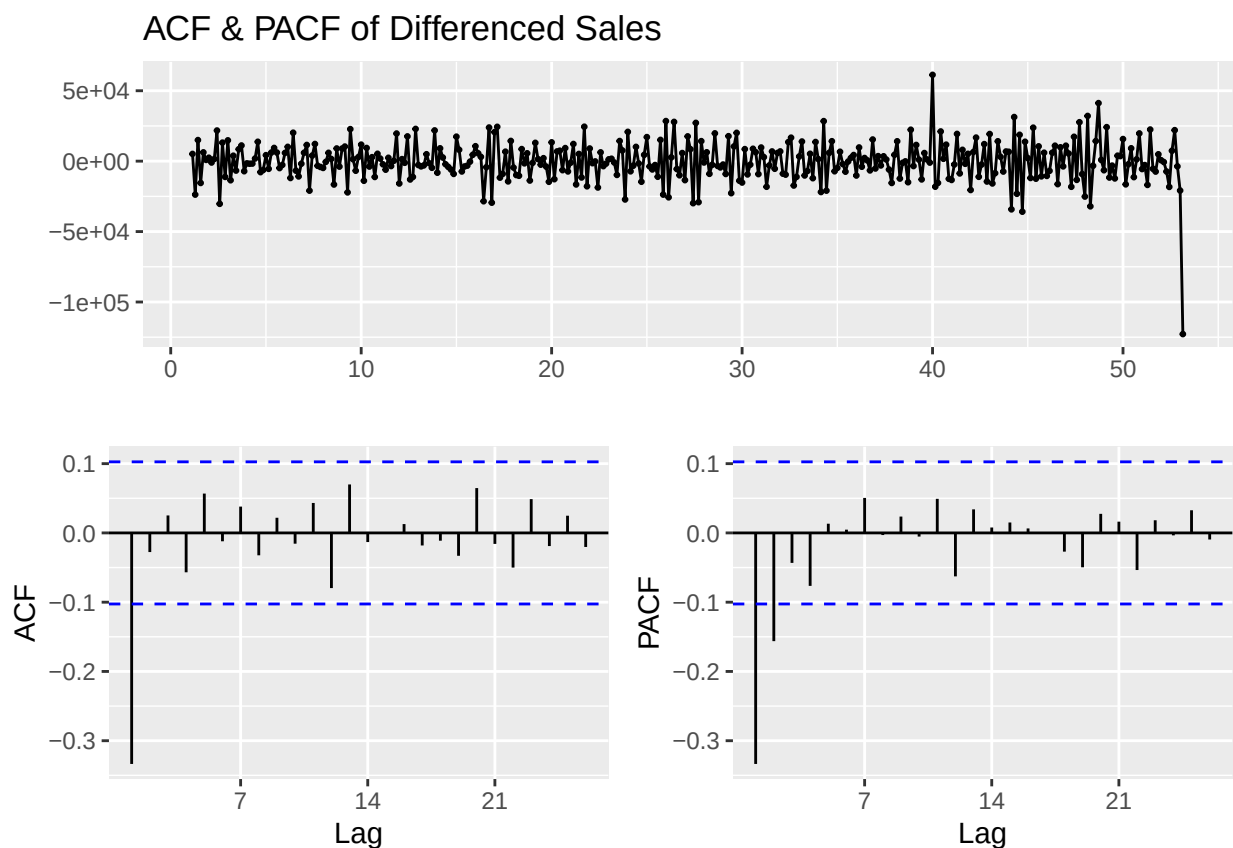
```
# use frequency = 7 >>> weekly seasonality
sales_ts2 <- ts(daily_sales$TotalSales, frequency = 7)
```

analyze Differencing

```
sales_diff <- diff(sales_ts2)
# we look at the Differenced data (Day 2 - Day 1)
```

plot ACF and PACF

```
ggttsdisplay(sales_diff, main="ACF & PACF of Differenced Sales")
```



MODEL BUILDING

Model A: (ARIMA 0,1,1)

```
model_a <- Arima(sales_ts2, order = c(0,1,1))
```

Model B: (ARIMA 1,1,1)

```
model_b <- Arima(sales_ts2, order = c(1,1,1))  
# checks if yesterday's sales value directly predicts today's
```

Model C: (SARIMA 1,1,1)(1,0,0)[7]

```
model_c <- Arima(sales_ts2,  
                 order = c(1,1,1),  
                 seasonal = list(order = c(1,1,1), period = 7))  
# looks for a 7-day (weekly) pattern
```

Compare Models

```
cat("AIC Comparison (note: Lower is Better):\n")
```

```
## AIC Comparison (note: Lower is Better):
```

```
cat("Model A (0,1,1):", AIC(model_a), "\n")
```

```
## Model A (0,1,1): 7950.204
```

```
cat("Model B (1,1,1):", AIC(model_b), "\n")
```

```
## Model B (1,1,1): 7952.06
```

```
cat("Model C (Seasonal):", AIC(model_c), "\n")
```

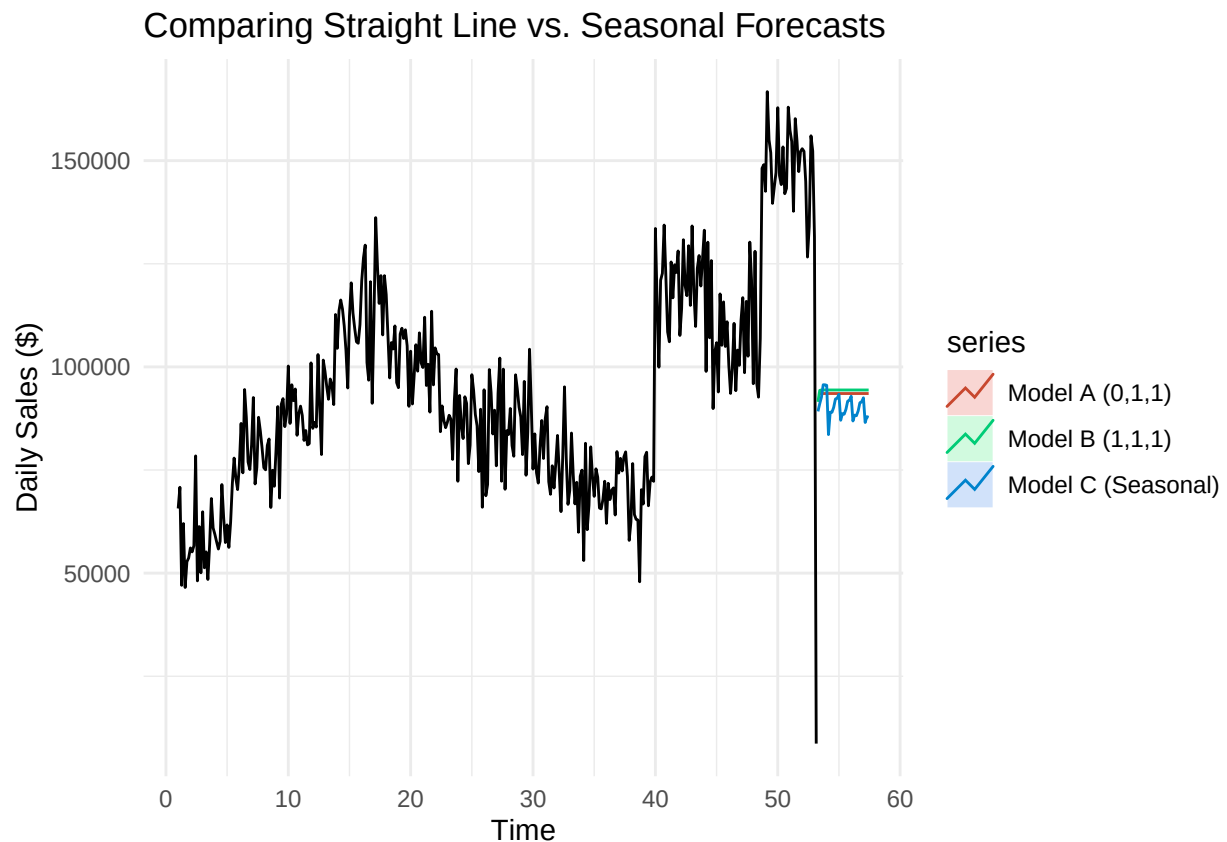
```
## Model C (Seasonal): 7833.535
```

Visual Comparison of Forecasts

```

autoplot(sales_ts2) +
  autolayer(forecast(model_a, h=30), series="Model A (0,1,1)", PI=FALSE) +
  autolayer(forecast(model_b, h=30), series="Model B (1,1,1)", PI=FALSE) +
  autolayer(forecast(model_c, h=30), series="Model C (Seasonal)", PI=FALSE) +
  labs(title = "Comparing Straight Line vs. Seasonal Forecasts",
       y = "Daily Sales ($)") +
  theme_minimal()

```

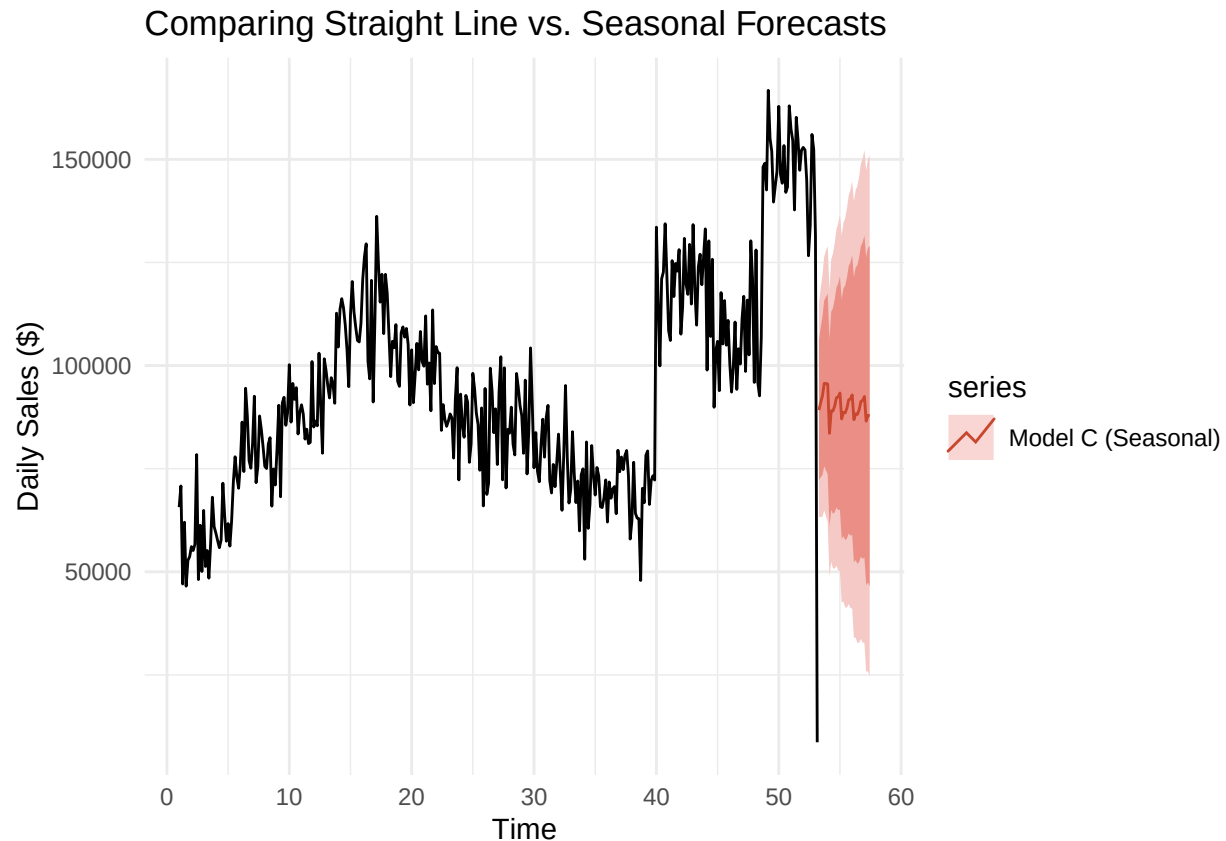


visualization with the uncertainty

```

autoplot(sales_ts2) +
  autolayer(forecast(model_c, h=30), series="Model C (Seasonal)", PI=TRUE) +
  labs(title = "Comparing Straight Line vs. Seasonal Forecasts",
       y = "Daily Sales ($)") +
  theme_minimal()

```



=== # Building Linear Model (Archive Part - only for review purpose) **Please note that we only add this part to show what we have done before Professor feedback during the presentation. We have changed our model to Time Series as mentioned above according to professor feedback**

1. Prepare Data: Aggregate sales by day

```
daily_sales2 <- all_sales_2019 %>%
  mutate(Date = as.Date(OrderDateTime, format = "%m/%d/%y %H:%M")) %>%
  group_by(Date) %>%
  summarize(TotalSales = sum(QuantityOrdered * PriceEach)) %>%
  mutate(DayIndex = as.numeric(Date - min(Date) + 1)) # Create a numeric time variable
```

2. Build Linear Model (Simple Linear Regression)

Independent Variable (X): Day of the year (Time). Dependent Variable (Y): Total Daily Sales. Goal: Determine if sales are significantly increasing or decreasing as the year progresses.

```
# Model: TotalSales = Intercept + Slope * DayIndex
sales_model <- lm(TotalSales ~ DayIndex, data = daily_sales2)
```

3. Model Evaluation

```
model_summary <- summary(sales_model)
print(model_summary)

##
## Call:
## lm(formula = TotalSales ~ DayIndex, data = daily_sales2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -108849  -16947    504    15898   52780
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 70834.64    2341.33   30.25  <2e-16 ***
## DayIndex    127.55      11.06   11.54  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 22350 on 364 degrees of freedom
## Multiple R-squared:  0.2677, Adjusted R-squared:  0.2657
## F-statistic: 133.1 on 1 and 364 DF, p-value: < 2.2e-16
```

Model Evaluation Intercept(70834.64): The model estimates that on Day 0 (the very start of the year), the business would have made about \$70834.64 in sales. **DayIndex(127.55):** For every single day that passes in 2019, the total sales increased by an average of \$127.55. Since this number is positive, the business is growing daily throughout the year.

P-Value: The P-Value is extremely small, showing statistically significant. Because the value is less than 0.05, we are confident that the increase in sales is real and not just due to random luck.

R-Squared Value

```
# Extract R-squared to see how well the date explains sales variance
r_squared <- model_summary$r.squared
cat("R-squared:", r_squared, "\n")
```

```
## R-squared: 0.2677058
```

While there is a clear upward trend (sales increasing by ~\$127/day), the date alone only explains about 27% of the sales volume. The remaining 73% of variation is *likely due to weekly cycles (weekends vs weekdays) and holiday spikes*.

4. Visualization with Regression Line

```
ggplot(daily_sales2, aes(x = Date, y = TotalSales)) +
  geom_point(alpha = 0.5, color = "navyblue") +      # Scatter plot of actual data
  geom_smooth(method = "lm", color = "red") +        # The regression line
  labs(title = "Daily Sales Trend (2019)",
```

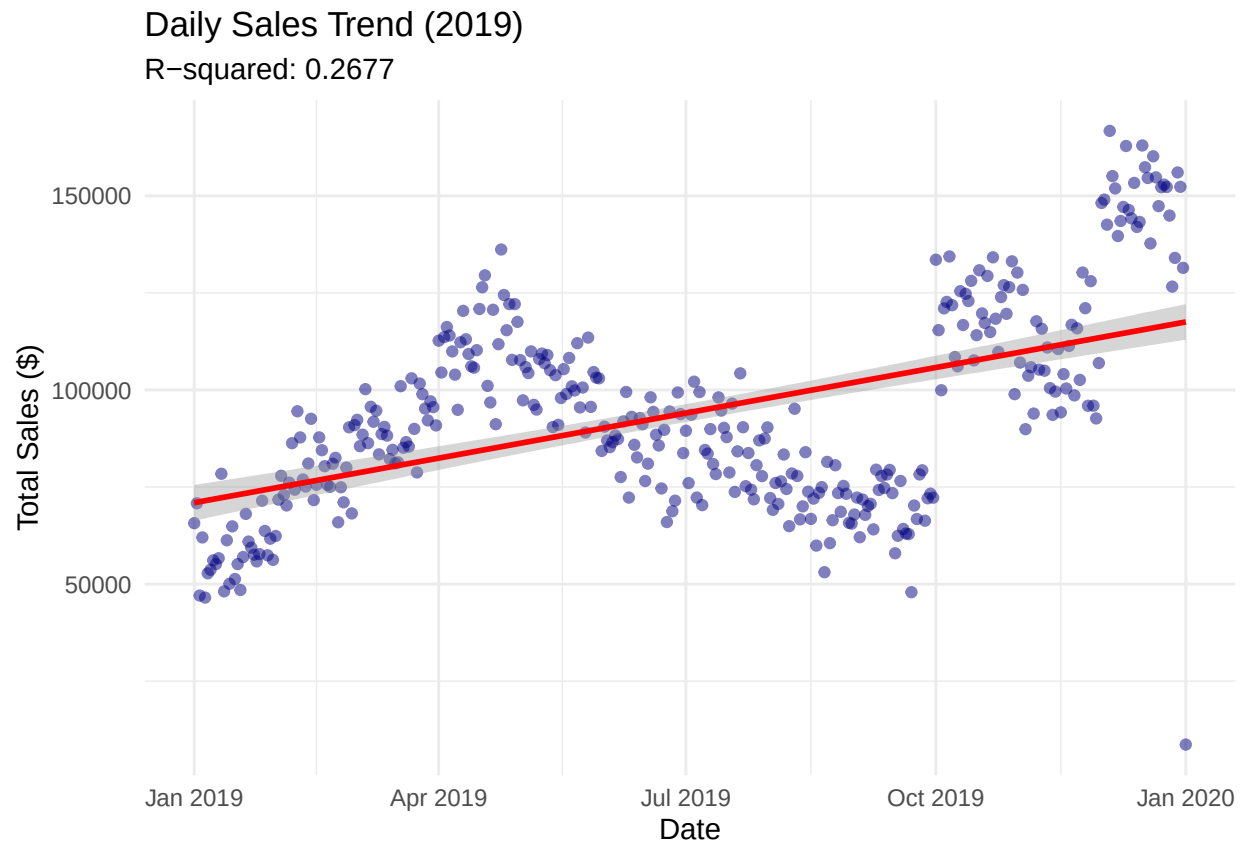


```

    subtitle = paste("R-squared:", round(r_squared, 4)),
    x = "Date",
    y = "Total Sales ($)" +
    theme_minimal()

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
# ggsave("outputs/figures/sales_trend_regression.png")
```

```
===
```